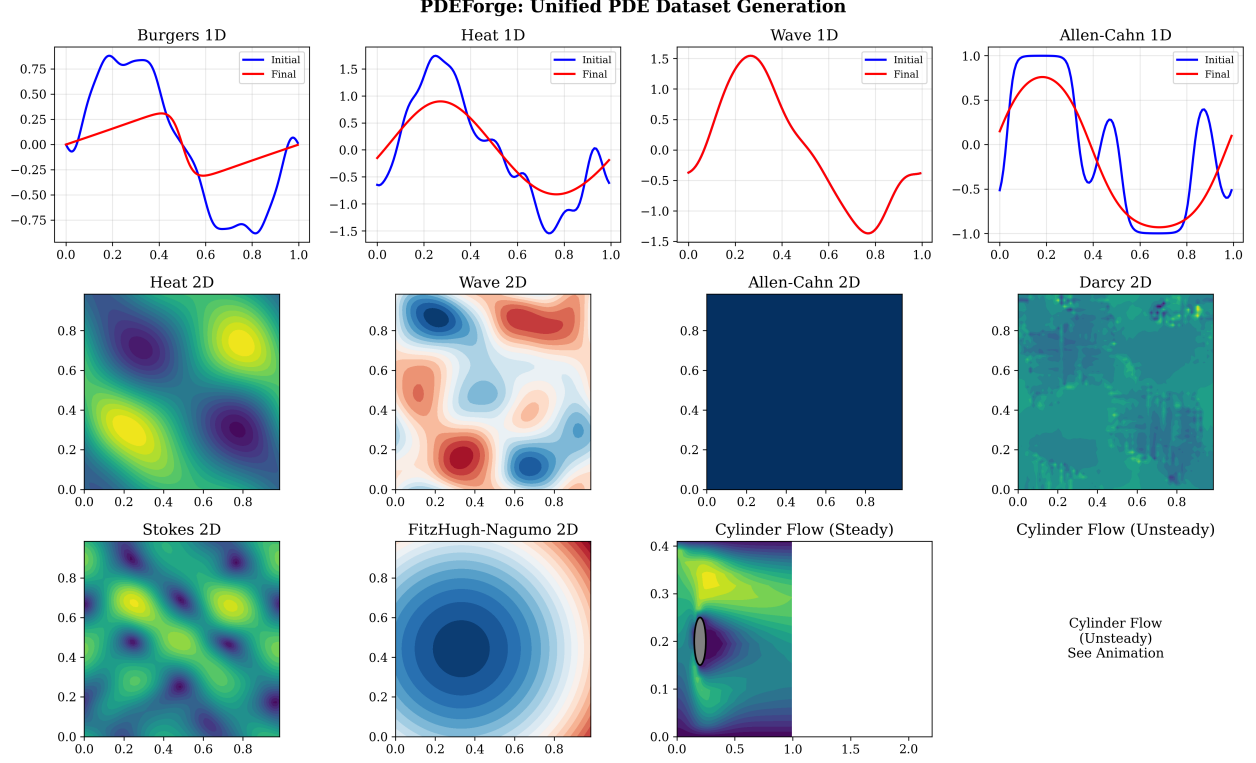


# PDEForge

Author: Peter Yatsyshin

A unified framework for generating PDE datasets for operator learning and uncertainty quantification.



PDEForge provides a simple, unified interface to generate training data for functional learning tasks from various PDE models. It is designed with **uncertainty quantification (UQ)** as a first-class concern.

## Why PDEForge?

| Challenge                             | PDEForge Solution                    |
|---------------------------------------|--------------------------------------|
| Static datasets with fixed resolution | Generate at any resolution you need  |
| Different APIs for each dataset       | Unified interface for all models     |
| No way to explore before downloading  | Interactive visualization            |
| No stochastic/UQ-ready datasets       | Built-in support for stochastic PDEs |
| Unclear train/val/test splits         | Dedicated calibration split for UQ   |

PDEForge is the data generation backbone for Operator\_UQ, our framework for uncertainty quantification in neural operators.

## Installation

```
# Basic installation
```

```
pip install pdeforge
```

```
# With notebook support (ipywidgets)
```

```
pip install pdeforge[notebook]
```

```
# With HDF5 support
pip install pdeforge[hdf5]

# All optional dependencies
pip install pdeforge[all]

# Development installation
git clone https://github.com/your-org/pdeforge.git
cd pdeforge
pip install -e ".[dev]"
```

## Quick Start with Setup Scripts

We provide setup scripts to create conda environments with all dependencies:

```
# Basic environment (spectral models only)
./setup_env.sh

# Environment with FEniCSx support (for complex geometry models)
./setup_fenicsx_env.sh
```

These scripts create conda environments, install dependencies, register Jupyter kernels, and run tests.

## Why Two Installation Options?

PDEForge offers two installation paths because of the trade-off between simplicity and capability:

|                         | Basic Install                | FEniCSx Install                        |
|-------------------------|------------------------------|----------------------------------------|
| <b>Dependencies</b>     | NumPy, SciPy, Matplotlib     | + FEniCSx, PETSc, MPI, gmsh            |
| <b>Install time</b>     | ~1 minute                    | ~1-5 minutes (with mamba)              |
| <b>Models available</b> | 13 spectral models           | 17 models (+ complex geometry)         |
| <b>Use case</b>         | Most operator learning tasks | Flow around obstacles, complex domains |

**Most PDE problems can be solved with spectral methods.** The basic installation covers:

- 1D: Burgers, Heat, Wave, Allen-Cahn, FitzHugh-Nagumo, Stochastic Heat
- 2D: Darcy flow, Stokes flow, Heat, Wave, Allen-Cahn, FitzHugh-Nagumo, Stochastic Heat

FEniCSx is only needed for **complex geometries** (e.g., flow around obstacles) where spectral methods don't apply. If you're unsure, start with the basic installation—you can always add FEniCSx later.

## Quick Start

```
from pdeforge import generate_dataset, list_models

# See available models
print(list_models())
# ['burgers_1d', 'darcy_2d', 'stokes_2d']

# Generate a dataset - same API for all models!
dataset = generate_dataset(
    model="burgers_1d",
    n_samples=1000,
    resolution={"x": 256},
```

```

    params={"viscosity": 0.01},
    seed=42,
)

# Explore the dataset
print(dataset)
# PDEDataset(
#   n_samples=1000,
#   input_shape=(256,),
#   output_shape=(256,),
#   input_names=['u0'],
#   output_names=['u_T'],
#   model=burgers_1d
# )

# Split into train/val/test
splits = dataset.split(train=0.6, val=0.15, cal=0.15, test=0.1)

# Interactive visualization (in Jupyter)
dataset.visualize()

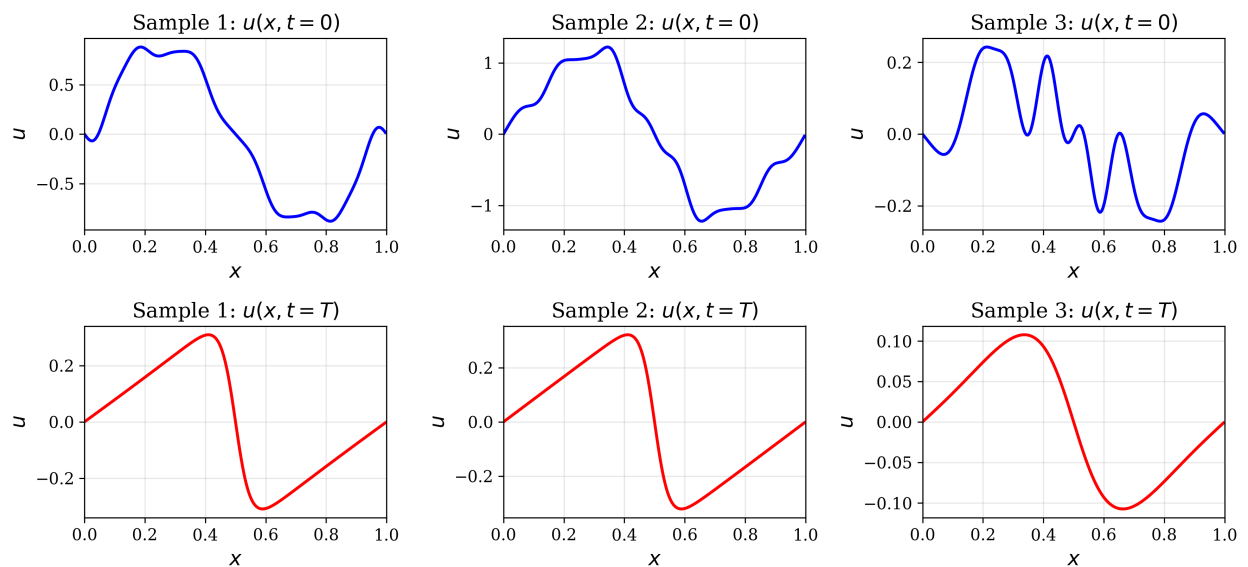
# Save for later
dataset.save("./my_dataset")

```

## Available Models

### 1D Burgers Equation (burgers\_1d)

#### Burgers Equation: Initial Conditions → Shock Formation



Advection-diffusion equation with shock formation:

$$\frac{\partial u}{\partial t} + \mu u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

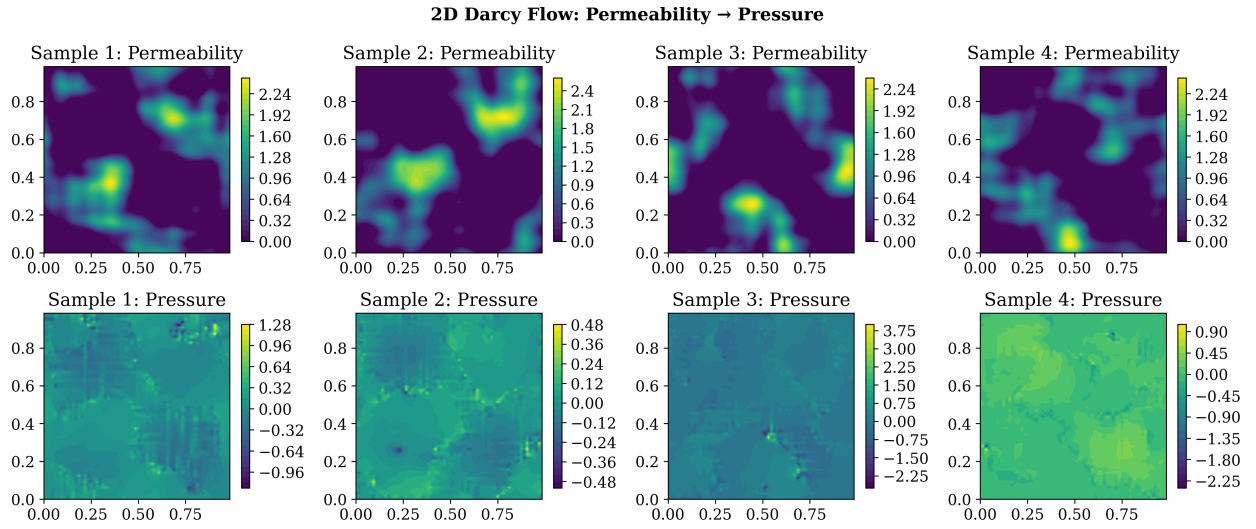
**Task:**  $u(x, t=0) \rightarrow u(x, t=T)$

```

dataset = generate_dataset(
    model="burgers_1d",
    n_samples=1000,
    resolution={"x": 256},
    params={
        "viscosity": 0.01, #
        "advection": 1.0, #
        "time_end": 1.0, # T
    },
)

```

## 2D Darcy Flow (darcy\_2d)



Steady-state flow through porous media:

$$-\nabla \cdot (\kappa(x, y) \nabla u) = f$$

**Task:**  $\kappa(x, y) \rightarrow u(x, y)$

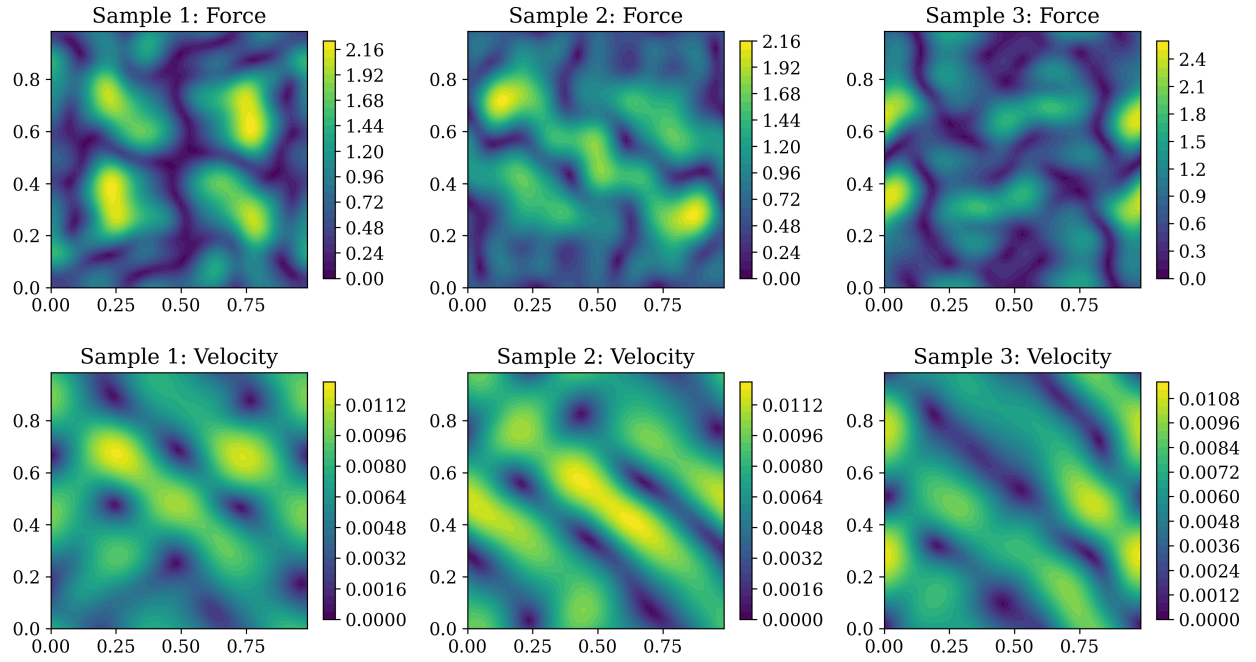
```

dataset = generate_dataset(
    model="darcy_2d",
    n_samples=1000,
    resolution={"x": 64, "y": 64},
    params={
        "kappa_min": 0.1, # Minimum permeability
        "kappa_max": 10.0, # Maximum permeability
    },
)

```

## 2D Stokes Flow (stokes\_2d)

### 2D Stokes Flow: Force → Velocity



Creeping viscous flow (low Reynolds number):

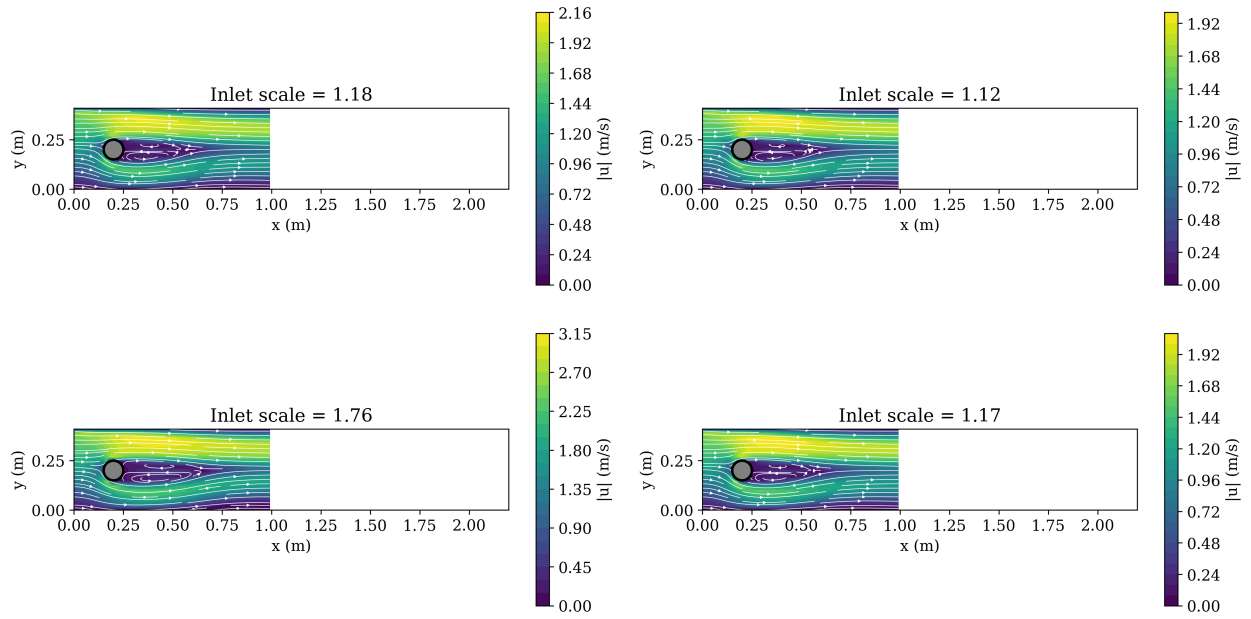
$$-\mu \nabla^2 \mathbf{u} + \nabla p = \mathbf{f}, \quad \nabla \cdot \mathbf{u} = 0$$

**Task:**  $(f_x, f_y) \rightarrow (u, v, p)$

```
dataset = generate_dataset(
    model="stokes_2d",
    n_samples=1000,
    resolution={"x": 64, "y": 64},
    params={
        "viscosity": 1.0,
        "n_force_modes": 5,
    },
)
```

## 2D Cylinder Flow (cylinder\_flow\_2d) - FEniCSx

### Steady Cylinder Flow: Velocity Magnitude with Streamlines



Flow around a circular cylinder (requires FEniCSx):

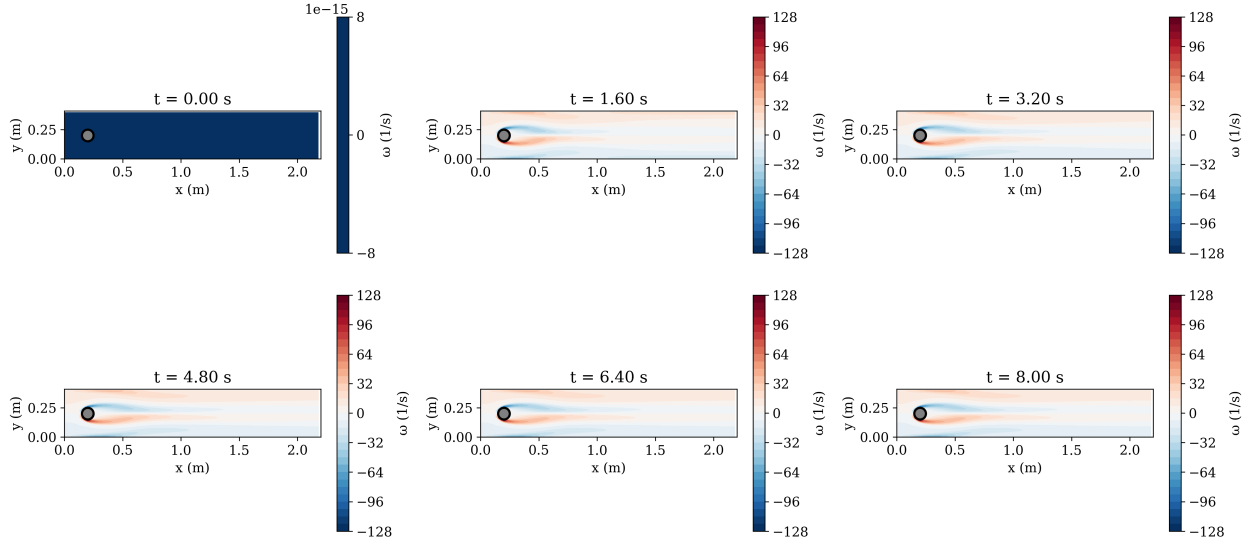
$$\rho(\mathbf{u} \cdot \nabla) \mathbf{u} - \mu \nabla^2 \mathbf{u} + \nabla p = 0, \quad \nabla \cdot \mathbf{u} = 0$$

**Task:** inlet velocity scale  $\rightarrow (u, v, p)$

```
dataset = generate_dataset(
    model="cylinder_flow_2d",
    n_samples=100,
    resolution={"x": 128, "y": 64},
    params={
        "viscosity": 0.001,
        "inlet_velocity": 0.3,
    },
)
```

## 2D Unsteady Cylinder Flow (cylinder\_flow\_2d\_unsteady) - FEniCSx

Unsteady Cylinder Flow: Vorticity (Von Kármán Vortex Street)



Time-dependent flow capturing **vortex shedding** (von Kármán vortex street):

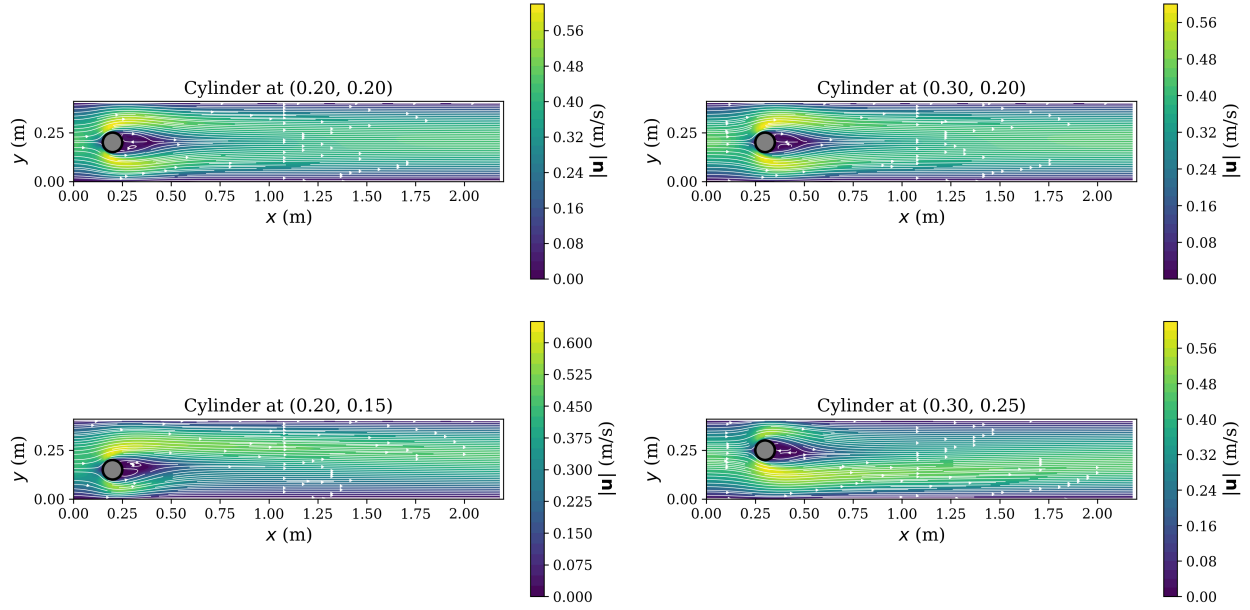
$$\rho \left( \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right) - \mu \nabla^2 \mathbf{u} + \nabla p = 0$$

**Task:** inlet velocity  $\rightarrow$  trajectory  $(u, v, p)(t)$  for  $t \in [0, T]$

```
dataset = generate_dataset(
    model="cylinder_flow_2d_unsteady",
    n_samples=5,
    resolution={"x": 110, "y": 41},
    params={
        "inlet_velocity": 1.0,
        "time_end": 8.0,
        "_n_time_steps": 81,
    },
)
# dataset.outputs.shape = (5, 81, 41, 110, 3) # (samples, time, y, x, channels)
```

## 2D Parameterized Cylinder Flow (cylinder\_flow\_2d\_parameterized) - FEniCSx

### Parameterized Cylinder Flow: Effect of Cylinder Position



Flow around a cylinder with **variable cylinder position**:

$$\rho(\mathbf{u} \cdot \nabla) \mathbf{u} - \mu \nabla^2 \mathbf{u} + \nabla p = 0, \quad \nabla \cdot \mathbf{u} = 0$$

**Task:** (inlet velocity,  $c_x$ ,  $c_y$ )  $\rightarrow$  ( $u$ ,  $v$ ,  $p$ )

This model allows the cylinder center position ( $c_x, c_y$ ) to vary across samples, enabling learning of flow patterns for different obstacle positions.

```
dataset = generate_dataset(
    model="cylinder_flow_2d_parameterized",
    n_samples=10,
    resolution={"x": 110, "y": 41},
    params={
        "inlet_velocity": 0.3,
        "cx_range": (0.2, 0.4),
        "cy_range": (0.15, 0.25),
    },
)
```

## 2D Turbulent Cylinder Flow (cylinder\_flow\_2d\_turbulent) - FEniCSx

High Reynolds number turbulent flow around a cylinder using **LES (Large Eddy Simulation)** with Smagorinsky subgrid-scale model:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} - \nabla \cdot ((\nu + \nu_t) \nabla \mathbf{u}) + \nabla p = 0$$

where  $\nu_t = (C_s \Delta)^2 |S|$  is the turbulent eddy viscosity.

**Task:** (inlet velocity,  $c_x$ ,  $c_y$ )  $\rightarrow$  ( $u$ ,  $v$ ,  $p$ )( $t$ ) time series



Features:

- SUPG/PSPG stabilization for convection-dominated flows
- Smagorinsky LES turbulence model
- Captures complex vortex dynamics at high Re

```
dataset = generate_dataset(  
    model="cylinder_flow_2d_turbulent",  
    n_samples=5,  
    resolution={"x": 220, "y": 82},  
    params={  
        "inlet_velocity": 1.0,  
        "viscosity": 0.0001, # Re ~ 1000  
        "use_les": True,  
        "time_end": 10.0,  
    },  
)
```

## Discover Available Models

```
from pdeforge import list_models, describe_model  
  
# List all models  
print(list_models()) # ['burgers_1d', 'darcy_2d', 'stokes_2d', ...]  
  
# Get detailed info about a model  
print(describe_model("burgers_1d"))  
# Shows: description, inputs/outputs, configurable parameters
```

## Unified API

All models share the same function signature:

```
dataset = generate_dataset(  
    model: str, # Model name  
    n_samples: int, # Number of samples  
    resolution: dict, # Grid resolution, e.g., {"x": 256} or {"x": 64, "y": 64}  
    domain: dict = None, # Domain bounds (default: unit domain)  
    params: dict = None, # Model-specific parameters  
    ic_generator: str = "fourier", # Initial condition generator  
    ic_params: dict = None, # IC generator parameters  
    seed: int = None, # Random seed for reproducibility  
    validate: bool = True, # Validate solutions  
    verbose: bool = True, # Show progress  
)
```

## Adding New Models

PDEForge uses a registry pattern to make adding new models easy:

```
from pdeforge.core.base import PDEModel  
from pdeforge.core.registry import register_model  
  
@register_model("my_custom_pde")  
class MyCustomPDE(PDEModel):  
    """My custom PDE model."""
```

```

NDIM = 2 # Spatial dimensions
DEFAULT_PARAMS = {"param1": 1.0}
INPUT_NAMES = ["input_field"]
OUTPUT_NAMES = ["output_field"]

def __init__(self, resolution, domain=None, **params):
    super().__init__(resolution, domain, **params)
    # Setup your model...

def solve(self, ic):
    """Solve the PDE given initial conditions."""
    # Implement your solver...
    return solution

def generate_ic(self, generator="fourier", generator_params=None, seed=None):
    """Generate random initial conditions."""
    # Use built-in generators or implement your own...
    return ic

# Now it's available!
dataset = generate_dataset(model="my_custom_pde", n_samples=100, ...)

```

## Parameter Exploration

Before generating large training datasets, explore how parameters affect the physics:

```
from pdeforge import explore_parameter, visualize_parameter_effect
```

```

# See how viscosity affects Burgers equation solutions
dataset = explore_parameter(
    model="burgers_1d",
    param_name="viscosity",
    param_values=[0.001, 0.01, 0.1],
    resolution={"x": 128},
    n_samples_per_value=3, # Same IC, different viscosity
)

```

```

# Visualize the effect
visualize_parameter_effect(dataset)

```

Or explore all parameters at once:

```

from pdeforge import explore_model

# Explore all user-facing parameters
explorations = explore_model("burgers_1d", resolution={"x": 128})

for param_name, dataset in explorations.items():
    print(f"Effect of {param_name}:")
    visualize_parameter_effect(dataset)

```

This is invaluable for:

- Building intuition about the physics before training
- Choosing appropriate parameter ranges for training data
- Understanding what makes certain parameter regimes "harder" to learn

## Interactive Visualization

PDEForge includes built-in visualization tools for Jupyter notebooks:

```
# Explore a generated dataset
from pdeforge.visualization import DatasetExplorer

explorer = DatasetExplorer(dataset)
explorer.show() # Interactive widget with sliders

# Preview a model before generating full dataset
from pdeforge import get_model

model = get_model("stokes_2d")(resolution={"x": 64, "y": 64})
model.preview(n_samples=3) # Quick preview
```

## Data Format

PDEDataset provides convenient access to your data:

```
# Access arrays
inputs = dataset.inputs      # Shape: (n_samples, *spatial_dims, n_input_channels)
outputs = dataset.outputs    # Shape: (n_samples, *spatial_dims, n_output_channels)
grid = dataset.grid          # Dict: {"x": array, "y": array, ...}

# Metadata
print(dataset.metadata)      # Model params, generation info

# Save/Load
dataset.save("./my_dataset")    # Directory format
dataset.save("./my_dataset.npz") # Compressed NPZ
dataset.save("./my_dataset.h5")  # HDF5

from pdeforge import load_dataset
loaded = load_dataset("./my_dataset")
```

## Use Cases

### 1. Standard Operator Learning

Learn mappings between function spaces:

```
# Initial condition → solution at time T
dataset = generate_dataset("burgers_1d", n_samples=10000, ...)

# Input field → output field
dataset = generate_dataset("darcy_2d", n_samples=5000, ...)
```

### 2. Uncertainty Quantification (UQ)

PDEForge datasets include a dedicated **calibration split** for conformal prediction and other UQ methods:

```
splits = dataset.split(train=0.6, val=0.15, cal=0.15, test=0.1)

# Train your neural operator on splits['train']
# Tune hyperparameters on splits['val']
# Calibrate uncertainty on splits['cal'] + For UQ!
# Final evaluation on splits['test']
```

### 3. Stochastic Systems (Planned)

For stochastic PDEs, we provide two output formats:

```
# Multiple realizations per input (for generative models)
dataset = generate_dataset(
    "stochastic_heat_2d",
    n_samples=1000,
    params={"n_realizations": 50}, # 50 noise realizations per IC
)
# dataset.outputs.shape = (1000, 50, nx, ny)

# Moment-based (for UQ: learn mean and variance)
dataset = generate_dataset(
    "stochastic_heat_2d",
    n_samples=1000,
    params={"output_moments": True},
)
# dataset.output_mean.shape = (1000, nx, ny)
# dataset.output_var.shape = (1000, nx, ny)
```

### 4. Parameter Sensitivity Studies

Understand how physical parameters affect solutions:

```
from pdeforge import explore_model

# Explore all parameters of a model
explorations = explore_model("burgers_1d", resolution={"x": 128})
for param, dataset in explorations.items():
    visualize_parameter_effect(dataset)
```

## Design Philosophy

PDEForge is for **generating ML training data**, not a general PDE solver library.

Models expose only parameters that affect data characteristics for machine learning:

| Exposed to Users                    | Hidden from Users             |
|-------------------------------------|-------------------------------|
| Physical parameters (viscosity, Re) | Solver tolerances             |
| Domain/geometry settings            | Mesh/discretization internals |
| Input field characteristics         | Time stepping details         |

Use `describe_model("model_name")` to see configurable parameters:

```
from pdeforge import describe_model
print(describe_model("burgers_1d"))
```

## Model Roadmap

Available Now: 17 Models

| Model      | Type                | Dimensions | Backend  |
|------------|---------------------|------------|----------|
| burgers_1d | Advection-diffusion | 1D         | Spectral |
| heat_1d    | Diffusion           | 1D         | Spectral |

| Model                          | Type                        | Dimensions | Backend  |
|--------------------------------|-----------------------------|------------|----------|
| heat_2d                        | Diffusion                   | 2D         | Spectral |
| wave_1d                        | Hyperbolic (oscillatory)    | 1D         | Spectral |
| wave_2d                        | Hyperbolic (oscillatory)    | 2D         | Spectral |
| allen_cahn_1d                  | Phase separation (bistable) | 1D         | Spectral |
| allen_cahn_2d                  | Phase separation (bistable) | 2D         | Spectral |
| fitzhugh_nagumo_1d             | Excitable media (neurons)   | 1D         | Spectral |
| fitzhugh_nagumo_2d             | Excitable media (spirals)   | 2D         | Spectral |
| darcy_2d                       | Elliptic (steady)           | 2D         | Spectral |
| stokes_2d                      | Incompressible flow         | 2D         | Spectral |
| stochastic_heat_1d             | Diffusion + noise           | 1D         | Spectral |
| stochastic_heat_2d             | Diffusion + noise           | 2D         | Spectral |
| cylinder_flow_2d               | Flow with obstacle (steady) | 2D         | FEniCSx  |
| cylinder_flow_2d_unsteady      | Vortex shedding (time-dep)  | 2D+t       | FEniCSx  |
| cylinder_flow_2d_parameterized | Variable obstacle position  | 2D         | FEniCSx  |
| cylinder_flow_2d_turbulent     | High-Re LES turbulence      | 2D+t       | FEniCSx  |

Use `describe_all_models()` for a quick overview:

```
from pdeforge import describe_all_models
print(describe_all_models())
```

## Stochastic Models

Stochastic models produce **multiple realizations per initial condition**:

```
dataset = generate_dataset(
    "stochastic_heat_1d",
    n_samples=100,
    params={"n_realizations": 20, "noise_intensity": 0.1},
)
# dataset.outputs.shape = (100, 20, nx) # 20 realizations per IC
```

Use for:

- **Generative models:** Learn conditional distributions  $P(u_T | u_0)$
- **Uncertainty quantification:** Estimate output variance from realizations

## Planned: Additional Models

| Model                    | Description                     | Status  |
|--------------------------|---------------------------------|---------|
| lotka_volterra_2d        | Predator-prey with diffusion    | Planned |
| gray_scott_2d            | Pattern formation               | Planned |
| stochastic_burgers_1d    | Burgers + noise                 | Planned |
| stochastic_allen_cahn_2d | Phase separation + fluctuations | Planned |

## Comparison with "the-well"

| Feature       | the-well              | PDEForge                   |
|---------------|-----------------------|----------------------------|
| Resolution    | Fixed                 | <b>Configurable</b>        |
| API           | Different per dataset | <b>Unified</b>             |
| Visualization | None                  | <b>Interactive widgets</b> |

| Feature         | the-well     | PDEForge                      |
|-----------------|--------------|-------------------------------|
| Extensibility   | Closed       | <b>Open registry</b>          |
| Data generation | Pre-computed | <b>On-demand</b>              |
| Parameters      | Fixed        | <b>Configurable</b>           |
| Documentation   | Sparse       | <b>Self-describing models</b> |

## Performance

PDEForge uses **NumPy/SciPy** for maximum compatibility. This is fast enough for most research:

| Use Case    | Samples | Time    | Our Advice                 |
|-------------|---------|---------|----------------------------|
| Exploration | 100     | seconds | Just run it                |
| Development | 1,000   | minutes | Fine for iteration         |
| Training    | 10,000+ | hours   | Generate once, save, reuse |

```
# Generate once, reuse forever
dataset = generate_dataset("burgers_1d", n_samples=10000, ...)
dataset.save("./my_training_data")
```

```
# Load instantly
dataset = load_dataset("./my_training_data")
```

**Future:** We plan to add an optional JAX backend for GPU acceleration. See `docs/performance.md` for details.

## Requirements

- Python  $\geq 3.8$
- NumPy  $\geq 1.20$
- SciPy  $\geq 1.7$
- Matplotlib  $\geq 3.4$
- tqdm  $\geq 4.60$

Optional:

- ipywidgets  $\geq 7.6$  (for interactive visualization)
- h5py  $\geq 3.0$  (for HDF5 export)
- FEniCSx (for complex geometry models like cylinder flow)

## Installing FEniCSx Support

FEniCSx models (like `cylinder_flow_2d` and `cylinder_flow_2d_unsteady`) require additional dependencies that must be installed via `conda/mamba`.

**Important: Use micromamba or mamba, not conda**

FEniCSx has complex dependencies (PETSc, MPI, etc.) that can cause conda's classic solver to hang for **hours**. We strongly recommend using **micromamba** or **mamba**, which solve dependencies in seconds instead.

### Option 1: micromamba (Recommended - fastest)

```
# Install micromamba (one-time setup, ~2 seconds)
curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest | tar -xvj -C ~ bin/micromamba

# Create the FEniCSx environment (~30-60 seconds!)
```

```
~/bin/micromamba create -n pdeforge-fenicsx python=3.11 \
    fenics-dolfinx petsc4py mpi4py gmsh pyvista ipykernel \
    -c conda-forge -y

# Activate and install PDEForge
eval "$(/bin/micromamba shell hook -s bash)"
micromamba activate pdeforge-fenicsx
pip install -e /path/to/PDEForge

# Register Jupyter kernel
python -m ipykernel install --user --name pdeforge-fenicsx --display-name "PDEForge (FEniCSx)"
```

### Option 2: mamba (if you have it installed)

```
# If mamba is available in your base environment
mamba create -n pdeforge-fenicsx python=3.11 \
    fenics-dolfinx petsc4py mpi4py gmsh pyvista ipykernel \
    -c conda-forge -y

conda activate pdeforge-fenicsx
pip install -e /path/to/PDEForge
python -m ipykernel install --user --name pdeforge-fenicsx
```

### Option 3: conda (Not recommended - very slow)

```
# This can take 30+ minutes or hang indefinitely
conda create -n pdeforge-fenicsx python=3.11 \
    fenics-dolfinx petsc4py mpi4py gmsh pyvista ipykernel \
    -c conda-forge -y
```

If you must use conda and it hangs on "Solving environment", cancel it (Ctrl+C) and use micromamba instead.

### Option 4: Use our setup script

```
cd /path/to/PDEForge
bash setup_fenicsx_env.sh # Note: uses conda, may be slow
```

### Verifying the installation:

```
import dolfinx
print(f"FEniCSx version: {dolfinx.__version__}")

from pdeforge import list_models
print([m for m in list_models() if 'cylinder' in m])
# Should show: ['cylinder_flow_2d', 'cylinder_flow_2d_unsteady']
```

See notebooks/02\_adding\_fenicsx\_models.ipynb for a tutorial on using and contributing FEniCSx models.

### Known Issue: Jupyter Kernel with FEniCSx

FEniCSx uses MPI for parallelization, which can cause issues when running notebooks via `jupyter nbconvert --execute` or similar batch execution. The kernel may time out during initialization due to MPI/PETSc conflicts.

### Workarounds:

1. **Run interactively:** Open notebooks in JupyterLab and run cells manually (usually works fine)
2. **Run as Python scripts:** Extract code from notebooks and run directly with `python`
3. **Add MPI environment variable:** Add this at the top of your notebook:

```
import os
os.environ["OMPI_MCA_opal_warn_on_missing_libcuda"] = "0"
```

This is a known limitation of FEniCSx in Jupyter environments, not a PDEForge issue. The spectral models (basic installation) work without any issues in all execution modes.

## Integration with Operator\_UQ

PDEForge is designed as the data generation layer for Operator\_UQ, our framework for uncertainty quantification in neural operators.

### Typical workflow:

```
# 1. Generate data with PDEForge
from pdeforge import generate_dataset

dataset = generate_dataset("burgers_1d", n_samples=10000, resolution={"x": 256})
splits = dataset.split(train=0.6, val=0.15, cal=0.15, test=0.1)
dataset.save("./burgers_data")

# 2. Train and calibrate with Operator_UQ
from operator_uq import FNO, ConformalPredictor

model = FNO(modes=16, width=64)
model.fit(splits['train'])

# Calibrate prediction intervals
predictor = ConformalPredictor(model)
predictor.calibrate(splits['cal']) # Uses the dedicated calibration split!

# 3. Make predictions with uncertainty
y_pred, intervals = predictor.predict(splits['test'], confidence=0.9)
```

## Contributing

We welcome contributions! See CONTRIBUTING.md for guidelines.

To add a new PDE model:

1. Create a new file in pdeforge/models/
2. Implement the PDEModel interface
3. Register with @register\_model("model\_name")
4. Add tests in tests/
5. Update documentation

## License

MIT License - see LICENSE for details.

## Citation

If you use PDEForge in your research, please cite:

```
@software{pdeforge2026,
  author = {Yatsyshin, Peter},
  title = {PDEForge: A Unified Framework for PDE Dataset Generation},
  year = {2026},
```



```
url = {https://github.com/your-org/pdeforge}  
}
```